

Yazılım Mimarisi: Kuruluş ve Girdi Varsayımları

ÖFY

September 2026

Özet

Bir yazılım sistemi, yazıldığı anda doğru kabul edilen bir dizi varsayıma dayanır. Bu varsayımların büyük kısmı belgelenmez; kodun yapısına, veritabanı kısıtlarına ve eksik bırakılan kontrollere gömülü hâlde bulunur. Varsayımlar iki gruba ayrılır: sistemin tasarımına ilişkin kuruluş varsayımları ve dışarıdan gelen veriye ilişkin girdi varsayımları. Hataların önemli bir bölümü kodun kendisinden değil, kodun dayandığı bir varsayımın zamanla geçersiz hâle gelmesinden kaynaklanır. Açık varsayımlar kolay bulunur; asıl risk, hiçbir yerde yazılı olmayan örtük varsayımlardadır. Örtük varsayımlar en çok, kontrol *yapılmayan* yerlerde saklıdır. Bu nedenle bir sisteme müdahale etmeden önce varsayımların çıkarılması, yapılacak değişikliğin neyi bozabileceğini önceden gösterir.

Giriş

Varsayım (*assumption*), bir yazılım bileşeninin kontrol etmeden doğru kabul ettiği koşuldur. Örneğin bir fonksiyon, kendisine gelen tarihin her zaman YYYY-MM-DD biçiminde olduğunu kabul ediyorsa, bu durum bir varsayımdır. Fonksiyon bu biçimi denetlemez; yalnızca buna göre davranır.

Varsayım kendi başına bir kusur değildir. Her sistem bir yerde sınır çizmek zorundadır ve her koşulu denetleyen bir sistem hem yavaş hem karmaşık olur. Sorun, varsayımın bilinmemesinde ve bozulduğunda fark edilmemesindedir.

Yaygın bir kabul, bir sistemi anlamak için kodun ne yaptığını okumanın yeterli olduğudur. Bu kabul eksiktir. Kod neyin yapıldığını gösterir; ancak neyin *hep doğru olduğu kabul edildiğini* doğrudan göstermez. Bir sisteme geliştirici olarak dahil olmadan önce bu ikinci bilginin de çıkarılması gerekir.

Bu yaklaşımın klasik bir örneği Ariane 5 roketinin 1996'daki ilk uçuşudur. Ariane 4 için yazılmış bir yazılım modülü, yatay hız değerinin belirli bir aralıkta kalacağını varsayıyordu. Ariane 5'in farklı uçuş profili bu aralığı aştı, sayısal dönüşüm taşma hatası verdi ve roket kalkıştan yaklaşık 40 saniye sonra imha edildi (Lions, 1996). Kod Ariane 4 için doğruduydu; bozulan şey varsayımıydı.

Bu not, varsayımları iki grupta ele alır ve bunların mevcut bir sistemde nasıl bulunabileceğini açıklar. Belge yazma, ekip içi iletişim ve varsayımların tasarım aşamasında nasıl belgeleneceği kapsam dışıdır.

Kuruluş Varsayımları

Kuruluş varsayımları, sistemin hangi koşullar için tasarlandığına ilişkin kabullerdir. Bu varsayımlar sistemin genel yapısını belirler ve genellikle ilk tasarım kararlarıyla birlikte yerleşir.

Garlan, Allen ve Ockerbloom (1995), farklı bileşenlerin bir araya getirilmesindeki güçlüğün temel nedeninin, her bileşenin çevresine dair birbiriyle uyuşmayan varsayımlar taşıması olduğunu göstermiştir. Yazarlar bu durumu mimari uyumsuzluk (*architectural mismatch*) olarak adlandırmıştır.

Ölçek

Sistem belirli bir kullanıcı sayısı, veri hacmi ve trafik düzeyi için tasarlanır. Bin kayıt için yazılmış ve tüm tabloyu belleğe yükleyen bir sorgu, kayıt sayısı on milyona ulaştığında çalışmaz hâle gelir. Kod değişmemiştir; değişen, ölçeğe ilişkin varsayımdır.

Çalışma Ortamı

Bu grup; işletim sistemini, saat dilimini, dil ve bölge ayarlarını (*locale*) ve sistemin tek sunucuda mı yoksa birden fazla sunucuda mı çalıştığını kapsar. Tek sunucuda çalışan bir sistem, bellekte tutulan durumun herkes için ortak olduğunu varsayabilir. Sistem ikinci bir sunucuya dağıtıldığında bu varsayım bozulur ve iki sunucu farklı durumları görmeye başlar.

Zaman ve Sıra

Bazı sistemler olayların belirli bir sırayla geleceğini varsayar: sipariş kaydı ödmeden önce oluşur, gece işi gündüz işinden sonra çalışır. Ağ gecikmesi ya da yeniden deneme (*retry*) mekanizmaları bu sırayı bozabilir.

Dış Bağımlılıklar

Sistem, bağlı olduğu dış servislerin her zaman erişilebilir olduğunu ve hızlı yanıt verdiğini varsayabilir. Deutsch'un dağıtık sistemlere ilişkin yanılgılar listesi bu türden varsayımları sıralar; ağır güvenilir olduğu ve gecikmenin sıfır olduğu kabulleri bunların ilk ikisidir (Deutsch, 1994).

Veri Modeli

Veri modeli, iş kurallarına dair kabulleri taşır. “Her kullanıcının bir e-posta adresi vardır” ya da “bir kayıt hiçbir zaman silinmez, yalnızca pasif hâle getirilir” gibi kurallar bu gruba girer. Bu kurallar değiştiğinde, onlara dayanan sorgular sessizce yanlış sonuç üretmeye başlar.

Kullanıcı Tipi

Sistemi kimin kullandığı, hata toleransını belirler. Teknik bir ekibin kullandığı bir araç, kullanıcının hatalı girdi vermeyeceğini varsayabilir. Aynı araç son kullanıcıya açıldığında bu varsayım geçersiz olur.

Girdi Varsayımları

Girdi varsayımları, sistemin dışarıdan aldığı veriye nasıl baktığına ilişkin kabullerdir. Dışarıdan gelen veri; kullanıcı formları, API istekleri, dosyalar ve diğer sistemlerden gelen mesajlar olabilir.

Meyer (1992), bir bileşenin çağırılardan beklediği koşulları ön koşul (*pre-condition*) olarak adlandırmış ve bunların açıkça yazılmasını önermiştir. Girdi varsayımları, yazılmamış ön koşullar olarak düşünülebilir.

Biçim

Tarihin hangi formatta geldiği, ondalık ayırıcının nokta mı virgül mü olduğu ve dosyanın hangi türde olduğu bu gruba girer. Türkçe bölge ayarlarında 3,5 olarak yazılan bir sayı, noktayı ayırıcı kabul eden bir sistemde hatalı okunur.

Değer Aralığı

Bir değer negatif olup olamayacağı, üst sınırının bulunup bulunmadığı bu grupta yer alır. Ariane 5 örneği, aralık varsayımının bozulmasının sonuçlarını gösterir (Lions, 1996).

Karakter Kodlaması

Sistem, metnin UTF-8 kodlamasıyla geldiğini ve belirli bir uzunluğu aşmadığını varsayabilir. Türkçe karakterler, emoji ve uzun isimler bu varsayımları sık bozan girdilerdir. Örneğin `VARCHAR(50)` olarak tanımlanmış bir alan, 50 karakteri aşan bir ismi kesebilir ya da reddedebilir.

Güven Düzeyi

Girdinin arka uçta denetlenip denetlenmediği önemli bir sorudur. Bazı sistemler, “ön yüz zaten kontrol ediyor” kabulüyle arka uçta denetim yapmaz. Bu durumda doğrudan API’ye gönderilen bir istek tüm denetimleri atlar.

Eksik Veri

Bir alan hiç gönderilmediğinde sistemin nasıl davrandığı çoğu zaman belirsizdir. Boş metin (“”) ile `null` değerinin aynı mı yoksa farklı mı kabul edildiği de bu grupta yer alır. Go dilinde bir `string` alanı gönderilmediğinde boş metin değerini alır; bu nedenle “alan gönderilmedi” ile “alan boş gönderildi” ayırt edilemez.

Tekrarlanan İstekler

Aynı isteğin iki kez gelmesi durumunda sistemin ne yapacağı bu gruba girer. Bir ödeme isteği ağ hatası nedeniyle yeniden gönderilirse, sistem iki ödeme mi yoksa

tek ödeme mi kaydeder? Aynı işlemin tekrarlanması sonucunu değiştirmemesi özelliğine **eş güçlülük** (*idempotency*) denir.

Hoşgörü İlkesi ve Sınırı

Postel'in hoşgörü ilkesi (*robustness principle*), bir sistemin gönderdiği veride katı, aldığı veride hoşgörü olmasını önerir (Postel, 1980). Bu ilke uyumluluğu artırır; ancak hoşgörüyle kabul edilen hatalı girdiler zamanla yeni örtük varsayımlara dönüşür. Karşı taraf, hatalı biçimin kabul edildiğini gördükçe ona güvenmeye başlar.

Varsayımların Bulunması

Varsayımların çoğu yazılı değildir. Bunları çıkarmak için kodun ve çevresinin belirli yerlerine bakılır.

Açık Varsayımların Kaynakları

- **Doğrulama kodu ve hata mesajları.** Hangi girdinin reddedildiği, kabul edilenin sınırını gösterir.
- **Veritabanı kısıtları.** NOT NULL, UNIQUE, yabancı anahtar (*foreign key*) ve alan uzunlukları, veri modeline ilişkin kabulleri doğrudan taşır.
- **Yapılandırma dosyaları ve varsayılan değerler.** Zaman aşımı süreleri, bağlantı havuzu boyutları ve sayfa limitleri ölçek varsayımlarını gösterir.

Örtük Varsayımların İzleri

- **Kontrol bulunmayan yerler.** Bir değer *if* ile denetlenmeden kullanıldığı her yerde, o değer “hep geçerli” olduğu varsayılmıştır.
- **Testler.** Testler neyin denendiğini gösterir; test edilmeyen durumlar ise hiç düşünülmemiş olabilir.
- **Commit geçmişi ve hata kayıtları.** Bozulan bir varsayım genellikle bir hata düzeltmesi olarak iz bırakır.
- **Sistemi uzun süredir tanıyan kişiler.** Belgelenmemiş kuralların önemli bir kısmı yalnızca kişilerin bilgisinde bulunur.

Davranışı Kayıt Altına Alma

Feathers (2004), eski bir sistemde değişiklik yapmadan önce sistemin mevcut davranışını kaydeden testler yazılmasını önermiştir. Bu testlere karakterizasyon testi (*characterization test*) denir. Bu testler neyin doğru olduğunu değil, sistemin şu anda ne yaptığını kaydeder. Böylece örtük varsayımlar gözlemlenebilir hâle gelir.

Kullanıcıların Dayandığı Davranış

Wright’ın gözlemine göre, yeterince çok kullanıcısı olan bir sistemde gözlemlenebilen her davranışa birileri bağımlı hâle gelir; belgede ne vaat edildiği fark etmez (Wright, t.y.). Bu gözlem Hyrum yasası (*Hyrum’s Law*) olarak bilinir. Buna göre varsayımlar yalnızca sistemin içinde değil, sistemi kullananların tarafında da bulunur.

Sonuç

Bir sistemin nasıl çalıştığını anlamak ile neye dayanarak çalıştığını anlamak ayrı işlerdir. İlki kodun okunmasıyla elde edilir; ikincisi kodun neyi denetlemediğinin incelenmesini gerektirir. Bir değişikliğin etkisini öngörmek için ikinci bilgi daha belirleyicidir. Değişiklik, bir varsayımı bozuyorsa o varsayıma dayanan bütün kod parçaları risk altına girer.

Pratik karşılığı şudur: bir değişiklik yapılmadan önce “bu kod neyin her zaman doğru olduğunu kabul ediyor?” sorusu yanıtlanmalıdır. Bu soru, değişikliğin kapsamını dokunulan satırlardan, o satırların dayandığı kabullere genişletir.

Bu yaklaşımın sınırları vardır. Büyük bir sistemde tüm varsayımların çıkarılması pratik değildir; çıkarım genellikle değiştirilecek bölgeyle sınırlı tutulur. Ayrıca bazı varsayımlar ancak bozulduklarında görünür hâle gelir ve önceden bulunamaz. Hoşgörülü girdi işleme ile katı denetim arasındaki denge de tartışmalıdır: fazla hoşgörü yeni örtük varsayımlar üretir, fazla katılık uyumluluğu azaltır.

Açık kalan soru, varsayımların sistem yazılırken nasıl kayıt altına alınacağıdır. Sözleşmeye dayalı tasarım, tip sistemleri ve veritabanı kısıtları bu yönde araçlar sunar; ancak hangi varsayımın koda, hangisinin belgeye yazılması gerektiğine dair genel bir ölçüt bulunmamaktadır.

Kaynaklar

- Deutsch, P. (1994). *The Eight Fallacies of Distributed Computing*. Sun Microsystems (yayınlanmamış liste; ilk yedi madde Deutsch’a, sekizinci madde Gosling’e atfedilir).
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Garlan, D., Allen, R., & Ockerbloom, J. (1995). Architectural Mismatch: Why Reuse Is So Hard. *IEEE Software*, 12(6).
- Lions, J.-L. (1996). *Ariane 5 Flight 501 Failure: Report by the Inquiry Board*. European Space Agency.
- Meyer, B. (1992). Applying “Design by Contract”. *IEEE Computer*, 25(10).
- Postel, J. (1980). *DoD Standard Transmission Control Protocol*. RFC 761.
- Wright, H. (t.y.). *Hyrum’s Law*. hyrumslaw.com.